Reviewer Pack — Minimal Rank of Geometric Invariants in Representation Theor…

Entry c32e54ee0266 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 20:24:52 UTC

Minimal Rank of Geometric Invariants in Representation Theory vs ACC0 Circuit Width

Entry ID: `c32e54ee0266` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 20:24:52 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Geometric Invariant Theory) **Field B** (complexity object): Complexity Theory: ACC0 Circuit Complexity

Statement:

> {'sentence_1': 'For every representation of a finite group G, the minimal rank of the geometric invariant associated with G is Θ(log n), where n is the size of the representation.', 'sentence_2': 'This invariant, when computed for a specific representation, provides an upper bound on the width of an ACC0 circuit that computes it.', 'sentence_3': 'For all representations with rank less than log n, there exists an ACC0 circuit of width exactly log n that computes the associated geometric invariant.'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Representation theory provides a rich source of algebraic structures and invariants that could potentially reveal hidden complexity-theoretic properties.', 'sentence_2': 'Geometric invariants, particularly those associated with group representations, have been studied for their intrinsic mathematical beauty but may also hold practical significance.', 'sentence_3': 'This conjecture aims to establish a direct connection between the algebraic structure of representation theory and circuit complexity, potentially leading to new insights into ACC0 circuit lower bounds.'}

Taxonomy category: `Geometric_Invariant_Representation_Theory_to_ACC0` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `485fc0f51308db25`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean width of ACC0 circuits for geometric invariants with ranks less than log n is within ±3 of log n, across all representations analyzed using 30 seeds. It is falsified if any seed produces a circuit width outside this range.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Minimal Rank Geometric Invariant Representation Theory AND ACC0 Circuit Width` - `Geometric Invariant Theory ACC0 Circuit Complexity relationship` - `ACC0 Circuit Width bounds Representation Theory geometric invariants`

Top relevant hits considered: - [http://arxiv.org/abs/1101.1408v3] Geometrizing the minimal representations of even orthogonal groups - [http://arxiv.org/abs/1101.2456v3] Polynomial representations and categorifications of Fock Space - [http://arxiv.org/abs/2105.12016v4] Waring ranks of sextic binary forms via geometric invariant theory - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/1611.00827v2] Geometric complexity theory and matrix powering - [http://arxiv.org/abs/1903.10609v3] Circuit Complexity of Knot States in Chern-Simons theory - [http://arxiv.org/abs/1603.02493v4] Clifford theory for glider representations - [http://arxiv.org/abs/0711.0795v4] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

```python
def matrix_power(M, k):
    n = len(M)
    result = [[0 if i != j else 1 for j in range(n)] for i in range(n)]
    while k > 0:
        if k % 2 == 1:
            result = matrix_multiply(result, M)
        M = matrix_multiply(M, M)
        k //= 2
    return result

def determinant(matrix):
    n = len(matrix)
    det = 0
    if n == 1:
        return matrix[0][0]
    elif n == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    else:
        for j in range(n):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += (-1) ** j * matrix[0][j] * determinant(submatrix)
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    G = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
    G = matrix_power(G, n - 1)

    det_G = determinant(G)
    rank_G = sum(1 for row in G if any(row))

    log_n = math.log2(n)

    if rank_G < log_n:
        circuit_width = int(log_n)
    else:
        circuit_width = None

    return {
        "metric_name": "Circuit Width",
        "metric_value": circuit_width,
        "instances_tested": 1,
        "conjecture_holds": circuit_width is not None and abs(circuit_width - log_n) <= 3,
        "counterexample": "" if circuit_width is not None else f"Rank {rank_G} < log({n}) = {log_r
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
    print(f"RESULT: FALSIFIED counterexample=\"Rank exceeds log(n)\" first_failing_seed={first
else:
    print("RESULT: INCONCLUSIVE")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Re-run the test with increased time limits to ensure it completes and produces results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14881
2	preregistration	ollama_remote	glm4:latest	0	0	9105
3	novelty	ollama_remote	glm4:latest	0	0	8247
4	novelty	ollama_remote	glm4:latest	0	0	9510
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15858
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12609
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16638
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9884
9	judge	ollama_remote	glm4:latest	0	0	11465

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 108195 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c32e54ee0266.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c32e54ee0266.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c32e54ee0266.tar.gz` (if generated)